

ReactJS

useMemo Hook:

What is useMemo?

useMemo is used to memoize (cache) the result of an expensive calculation so that React doesn't recompute it on every render. useMemo remembers the result of a calculation and recalculates it only when its dependencies change.

Syntax:

```
const memoizedValue = useMemo(() => {  
  return someExpensiveCalculation();  
}, [dependencies]);
```

- The function runs on the first render.
- It runs again only if one of the dependencies changes.
- Otherwise, React returns the cached value.

Now, let's see a real example of expensive calculation: (not using useMemo)

Example:

App.jsx:

```
src > App.jsx > App  
1 import React, { useState } from 'react'  
2 function App() {  
3   const [count, setCount] = useState(0);  
4   //creating the expensive task  
5   function expensiveTask(num){  
6     console.log("Inside expensive task");  
7     for (let i = 0; i<=1000000000; i++) {}  
8     return num*2;  
9   }  
10  //calling that expensive task  
11  let doubleValue = expensiveTask(4);  
12  return (  
13    <div>  
14      <button onClick={() => setCount(count+1)}>  
15        Increment  
16      </button>  
17      <div>  
18        Count: {count}  
19      </div>  
20      <div>  
21        Double: {doubleValue}  
22      </div>  
23    </div>  
24  )  
25 }  
26 export default App
```

Output:

Increment

Count: 0

Double: 8

Output: when clicked on the button. The value of count will take some extra time now to increase.

Increment
Count: 1
Double: 8

Example: doing the same thing using the useMemo hook.

App.jsx:

```
src > App.jsx > default
1 import React, { useState, useMemo } from 'react';
2 function App() {
3   const [count, setCount] = useState(0);
4   const [input, setInput] = useState(0);
5   function expensiveTask(num) {
6     console.log("Inside expensive task");
7     for (let i = 0; i <= 1000000000; i++) {}
8     return num * 2;
9   }
10  const doubleValue = useMemo(() => expensiveTask(input), [input]);
11  return (
12    <div>
13      <button onClick={() => setCount(count + 1)}>
14        Increment
15      </button>
16      <div>Count: {count}</div>
17      <input
18        type="number"
19        value={input}
20        onChange={(e) => setInput(Number(e.target.value))}
21      />
22      <div>Double: {doubleValue}</div>
23    </div>
24  );
25 }
26 export default App;
```

doubleValue is a cached result of expensiveTask(input) that updates only when input changes.

This component has two pieces of state: count and input. The expensiveTask() function simulates a slow calculation by looping many times and then returns input * 2. The useMemo hook caches the result of this calculation in doubleValue and only recalculates it when input changes. So, when you click the Increment button and only count changes, the component re-renders but expensiveTask() does not run again, improving performance by avoiding unnecessary expensive computations.

Output:

Increment
Count: 0

Double: 0

Output: now count will increase quickly, while the double calculation will take time for sure.

Increment
Count: 4

Double: 16

Another example of useMemo:

App.jsx:

```
src > App.jsx > App
1 import './App.css';
2 import React, { useState, useMemo } from 'react';
3 function App() {
4   const [count, setCount] = useState(0);
5   const [item, setItem] = useState(10);
6   const multiCountMemo = useMemo(function multiCount() {
7     console.warn('multiCount');
8     return count * 5;
9   }, [count]);
10  return (
11    <div className="App">
12      <h1>useMemo Hook in React</h1>
13      <h2>Count: {count}</h2>
14      <h2>Item: {item}</h2>
15      <h2>{multiCountMemo}</h2>
16      <button onClick={() => setCount(count + 1)}>Update Count
17      </button>
18      <button onClick={() => setItem(item * 10)}>Update Item
19      </button>
20    </div>
21  );
22 }
23 export default App;
```

Output:

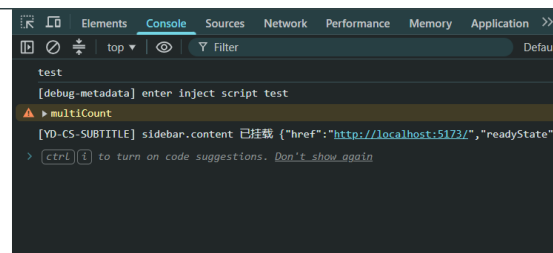
useMemo Hook in React

Count: 0

Item: 10

0

Update Count Update Item



Output: when Update count button was clicked multiple times. This is okay.

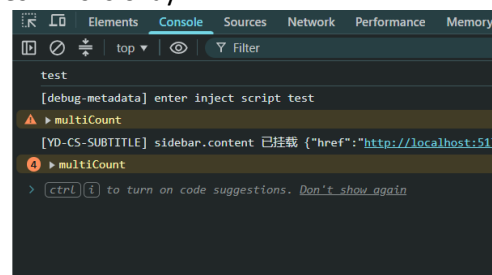
useMemo Hook in React

Count: 4

Item: 10

20

Update Count Update Item



Output: when update item is clicked. Clearly, it didn't lead to change the console.

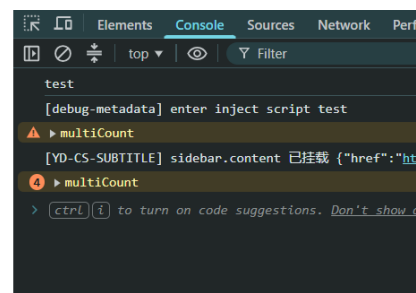
useMemo Hook in React

Count: 4

Item: 10000000

20

Update Count Update Item



Explanation: In this code, multiCountMemo is calculated using useMemo, which means React will only run the multiCount function again when count changes. So, when you click Update Count, count changes, multiCount runs again, and count * 5 is recalculated. However, when you click Update Item, only item changes, so React re-renders the component but does not execute multiCount again because count is still the same. If useMemo were not used and you simply wrote const multiCountMemo = count * 5 or called a function directly, then multiCount would run on every single re-render, including when only item changes. In this example the calculation is simple, so the performance difference is negligible, but for expensive computations useMemo helps avoid unnecessary recalculations and improves performance.

useCallback Hook:

What is useCallback?

useCallback is a built-in React Hook that lets you cache (memoize) a function definition between renders of a component.

Syntax:

```
const cachedFn = useCallback(fn, dependencies)
```

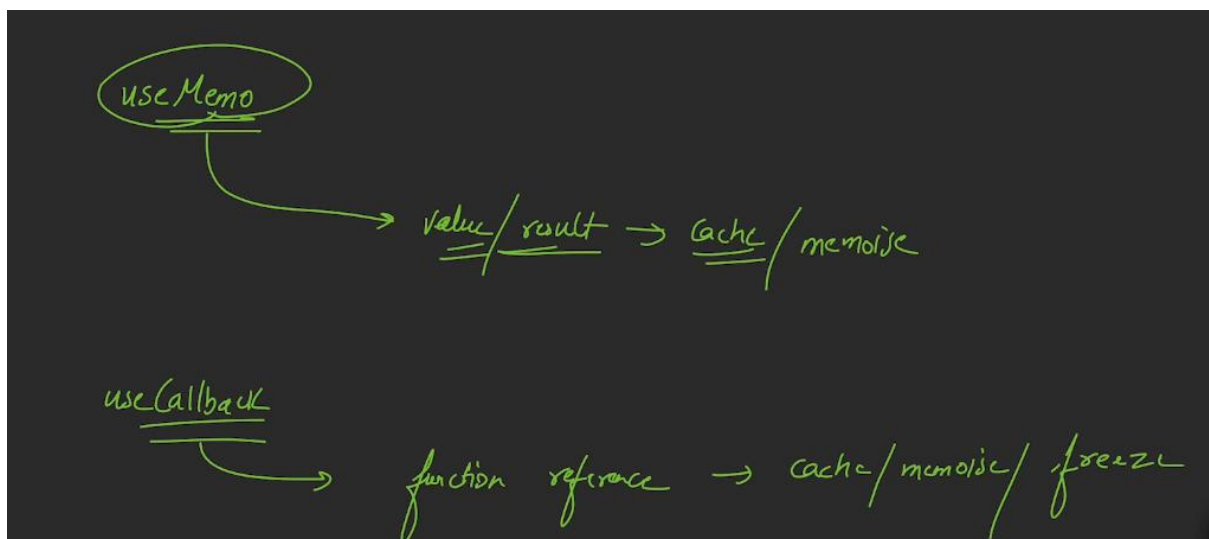
- fn: The function value that you want to cache.
- dependencies: An array of reactive values (props, state, or variables derived from them) used inside the function. React will recreate the function *only* if one of these dependencies changes.

Common Use Cases:

You should reach for useCallback primarily in these scenarios:

1. Passing callbacks to optimized child components: As shown above, when a child component relies on shallow prop comparison (React.memo) to prevent useless renders.
2. Custom Hooks: If you are writing a custom Hook and returning a function, wrapping it in useCallback ensures consumers of your hook can safely use it in their own dependency arrays.
3. Function is a dependency elsewhere: If your function is passed into *another* hook's dependency array (like a useEffect), omitting useCallback will cause that useEffect to run on every single render.

Basically,



Example: preventing the unnecessary re-rendering of child. But first see an example of unnecessary re-rendering of child.

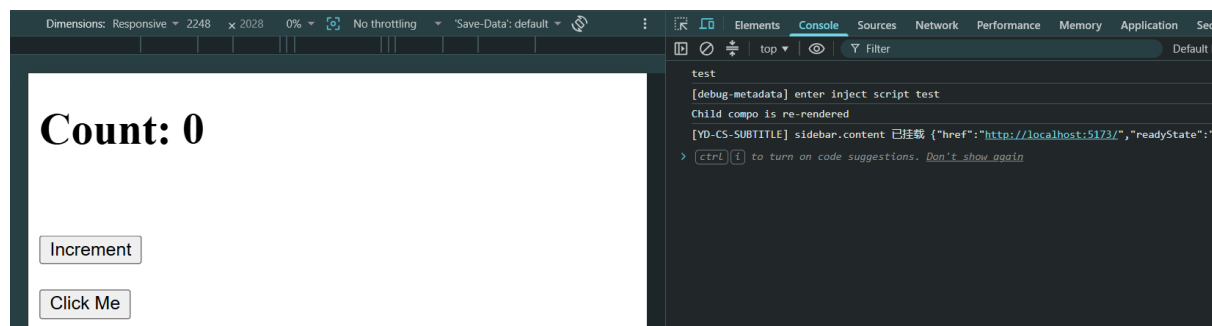
App.jsx:

```
src > App.jsx > default
1  import React, { useState } from 'react'
2  import ChildComponent from './myComponent/ChildComponent'
3  function App() {
4    const [count, setCount] = useState(0);
5    function handleClick(){
6      setCount(count+1);
7    }
8    return (
9      <div>
10       <h1>Count: {count}</h1>
11       <br/><br/>
12       <button onClick={handleClick}>Increment</button>
13       <br/><br/>
14       <ChildComponent buttonName="Click Me"/>
15     </div>
16   )
17 }
18 export default App
```

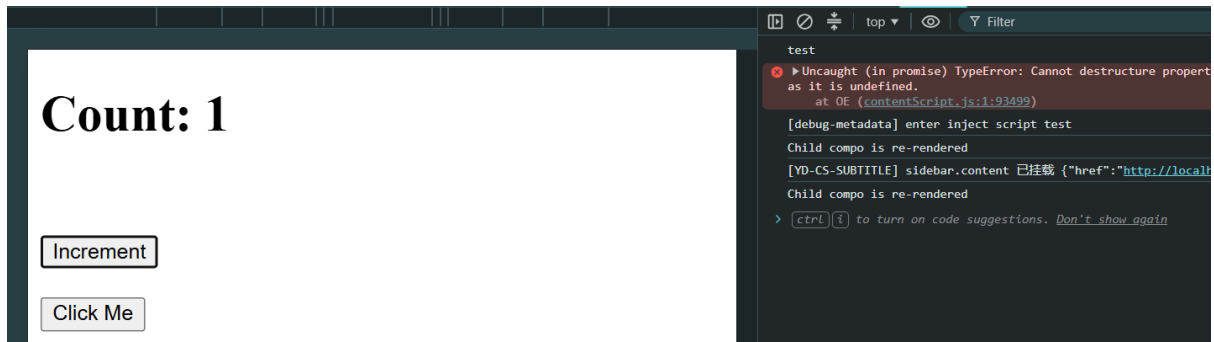
ChildComponent.jsx:

```
src > myComponent > ChildComponent.jsx > default
1  import React from 'react'
2
3  function ChildComponent(props) {
4    console.log("Child compo is re-rendered")
5    return (
6      <>
7        <button>
8          {props.buttonName}
9        </button>
10      </>
11    )
12  }
13
14  export default ChildComponent
```

Output: when loaded.



Output: when button increment button was clicked. Clearly, the number of times, the re-rendering happens the number of times, the ChildComponent get re-rendered.



What happens when you click Increment?

1. `setCount(count + 1)` updates the state.
2. App re-renders.
3. Since App re-renders, all of its children re-render by default, including ChildComponent.

So every time you click Increment, you'll see: Child compo is re-rendered in the console, even though `buttonName` never changes.

Why does this happen? React's default behavior is: When a parent component re-renders, all of its child components re-render too.

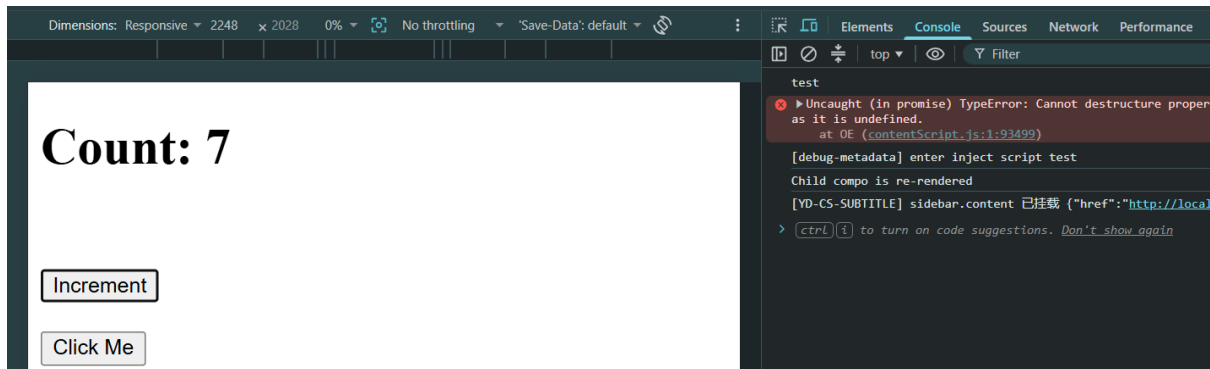
We can solve this issue using the `React.memo`. Wrap the child component with `React.memo`.

ChildComponent.jsx:

```
src > myComponent > ChildComponent.jsx > ...
1  import React from 'react'
2
3  function ChildComponent(props) {
4    console.log("Child compo is re-rendered")
5    return (
6      <>
7        <button>
8          {props.buttonName}
9        </button>
10     </>
11   )
12 }
13
14 export default React.memo(ChildComponent)
```

App.jsx: same as above.

Output: even if the Increment button was clicked multiple times, the console don't show that the child components was re-rendered again.



But is this the ideal solution? No! React.memo actually is preventing the re-rendering until a value is passed. But if we pass a function, then it will not prevent re-rendering.

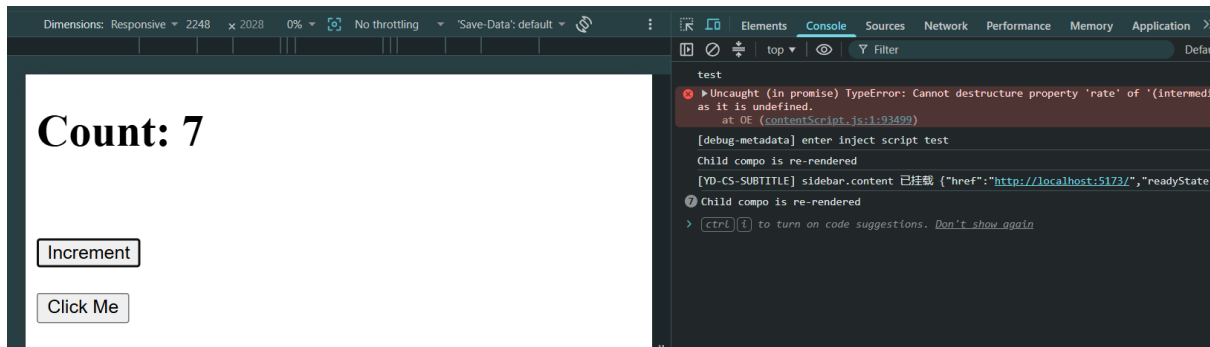
App.jsx:

```
src > App.jsx > App
1 import React, { useState } from 'react'
2 import ChildComponent from './myComponent/ChildComponent'
3 function App() {
4   const [count, setCount] = useState(0);
5   function handleClick(){
6     setCount(count+1);
7   }
8   return (
9     <div>
10       <h1>Count: {count}</h1>
11       <br/><br/>
12       <button onClick={handleClick}>Increment</button>
13       <br/><br/>
14       <ChildComponent buttonName="Click Me" handleClick={handleClick}/>
15     </div>
16   )
17 }
18 export default App
```

ChildComponent.jsx:

```
src > myComponent > ChildComponent.jsx > ...
1 import React from 'react'
2
3 function ChildComponent(props) {
4   console.log("Child compo is re-rendered")
5   return (
6     <>
7       <button onClick={props.handleClick}>
8         {props.buttonName}
9       </button>
10     </>
11   )
12 }
13
14 export default React.memo(ChildComponent)
```


Output: clearly, even if we used the React.memo in ChildComponent still the re-rendering is going on.



Basically, Every time App re-renders, this function is recreated, since it is recreated thus for every creation there is a new reference, because the function reference changes on every render. So React.memo says: "The props changed, so I need to re-render."

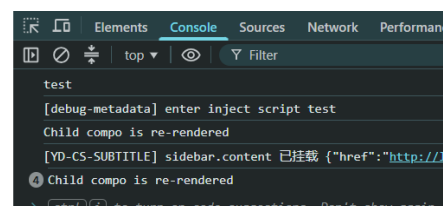
So, How to fix it? Use useCallback to memoize the function.

App.jsx:

```
src > App.jsx > App
1  import React, { useCallback, useState } from 'react'
2  import ChildComponent from './myComponent/ChildComponent'
3  function App() {
4    const [count, setCount] = useState(0);
5    const handleClick = useCallback(() => {
6      setCount(count+1);
7    }, [count]);
8    return (
9      <div>
10         <h1>Count: {count}</h1>
11         <br/><br/>
12         <button onClick={handleClick}>Increment</button>
13         <br/><br/>
14         <ChildComponent buttonName="Click Me" handleClick={handleClick}/>
15       </div>
16     )
17   }
18   export default App
```

Output: Callback depending on state, and since we passed count as the dependency, thus for every click as the count is changing, the re-rendering happening, and thus here also function is recreated. ChildComponent will re-render on every count change because handleClick gets a new reference every time count changes.

Count: 4



Let's see another example to understand concept of useCallback:

App.jsx:

```
src > App.jsx > default
1  import React, { useState, useCallback } from 'react';
2  import ChildA from './myComponent/ChildA';
3  function App() {
4    const [add, setAdd] = useState(0);
5    const Learning = useCallback(() => {
6      // some operation
7      console.log('Learning function called');
8    }, []);
9    return (
10     <div className="App">
11       <h1>Learning useCallback</h1>
12       <ChildA Learning={Learning} />
13       <h1>{add}</h1>
14       <button onClick={() => setAdd(add + 1)}>
15         Addition
16       </button>
17     </div>
18   );
19 }
20 export default App;
```

This component demonstrates the use of useCallback. The add state is updated when the Addition button is clicked, causing App to re-render. The Learning function is wrapped inside useCallback([]), so its reference remains the same across re-renders. Since Learning is passed to ChildA, which is wrapped with React.memo, ChildA does not re-render unnecessarily when add changes. This improves performance by preventing extra renders of child components whose props haven't actually changed.

ChildA.jsx:

```
src > myComponent > ChildA.jsx > ChildA
1  import React from 'react';
2
3  function ChildA({ Learning }) {
4    console.log('ChildA rendered');
5    return (
6      <div>
7        <button onClick={Learning}>
8          Call Learning Function
9        </button>
10      </div>
11    );
12  }
13  export default React.memo(ChildA);
```

ChildA is a memoized component that receives a Learning function as a prop and calls it when the button is clicked. The console.log helps us check when the component re-renders. Since the component is wrapped with React.memo, it will only re-render if the Learning prop changes. If Learning is memoized using useCallback in the parent, ChildA avoids unnecessary re-renders and improves performance.

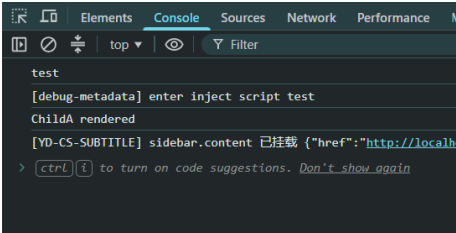
Output:

Learning useCallback

Call Learning Function

0

Addition



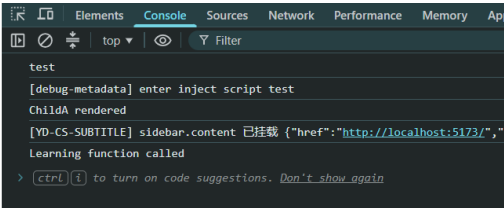
“call learning function” is called:

Learning useCallback

Call Learning Function

0

Addition



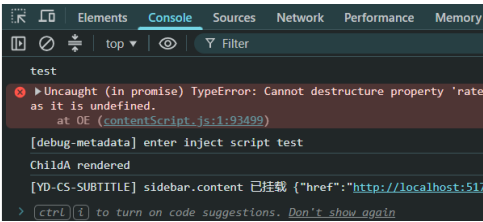
“Addition” button was clicked:

Learning useCallback

Call Learning Function

1

Addition



Hook	Returns	Use for
useMemo	A memoized value	Expensive calculations, objects, arrays
useCallback	A memoized function	Stable function references

Updating Objects in State:

How to approach this?

In React, state should be treated as immutable. That means when you want to update an object in state, you create a new object rather than modifying the existing one.

Let's first understand a basic thing (without object). We are trying to update the name, as given below:

App.jsx:

```
src > App.jsx > ...
1  import React, { useState } from 'react'
2
3  function App() {
4    const [name, setName] = useState('Aditya');
5    return (
6      <div>
7        <h2>{name}</h2>
8        <button onClick={()=>setName("Utsav")}>Update Name</button>
9      </div>
10   )
11 }
12
13 export default App
```

Output:

Aditya

Update Name

Output: when button was clicked. Name changed as expected.

Utsav

Update Name

This, can also be done by creating a separate function there. Right? Yes, very simple thing.

App.jsx:

```
src > App.jsx > App
1  import React, { useState } from 'react'
2  function App() {
3    const [name, setName] = useState('Aditya');
4    const handleName = ()=>{
5      setName("Utsav")
6    }
7    return (
8      <div>
9        <h2>{name}</h2>
10       <button onClick={handleName}>Update Name</button>
11     </div>
12   )
13 }
14 export default App
```

Output:

Aditya

Update Name

Output: when button was clicked. Name changed as expected.

Utsav

Update Name

Also, we know if there is an object or nested object in the useState then we can access it using ".", as shown below:

App.jsx:

```
src > App.jsx > default
1  import React, { useState } from 'react'
2  function App() {
3    const [data, setData] = useState({
4      name: 'Aditya',
5      address: {
6        city: 'Muzaffarpur',
7        country: 'India'
8      }
9    })
10   return (
11     <div>
12       <h2>Name: {data.name}</h2>
13       <h2>City: {data.address.city}</h2>
14       <h2>Country: {data.address.country}</h2>
15     </div>
16   )
17 }
18 export default App
```

Output:

Name: Aditya

City: Muzaffarpur

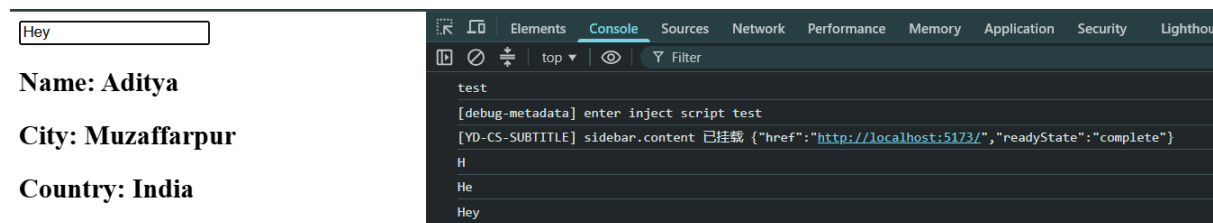
Country: India

Now, suppose there is an input field where user can enter the name, and we wish that name shall be displayed on the console. For that we will do like this: (we have covered this before as well)

App.jsx:

```
src > App.jsx > App
1 import React, { useState } from 'react'
2 function App() {
3   const [data, setData] = useState({
4     name: 'Aditya',
5     address: {
6       city: 'Muzaffarpur',
7       country: 'India'
8     }
9   })
10  const handleName=(val)=>{
11    console.log(val);
12  }
13  return (
14    <div>
15      <input type='text' placeholder='Type name'
16        onChange={(event)=>handleName(event.target.value)}/>
17      <h2>Name: {data.name}</h2>
18      <h2>City: {data.address.city}</h2>
19      <h2>Country: {data.address.country}</h2>
20    </div>
21  )
22 }
23 export default App
```

Output: the moment we started typing, it console'd that.



Now, suppose we want that the object shall also get changed, and reflected. For that we will use "...".

App.jsx:

```
src > App.jsx > App
1 import React, { useState } from 'react'
2 function App() {
3   const [data, setData] = useState({
4     name: 'Aditya',
5     address: {
6       city: 'Muzaffarpur',
7       country: 'India'
8     }
9   })
10  const handleName=(val)=>{
11    let tempData = data;
12    tempData.name = val;
13    setData({...tempData})
14  }
15  return (
16    <div>
17      <input type='text' placeholder='Type name'
18        onChange={(event)=>handleName(event.target.value)}/>
19      <h2>Name: {data.name}</h2>
20      <h2>City: {data.address.city}</h2>
21      <h2>Country: {data.address.country}</h2>
22    </div>
23  )
24 }
25 export default App
```

In handleName, we first store the current state object (data) in a temporary variable tempData, then update its name property with the new value entered by the user. After that, we use setData({ ...tempData }) to create a new object and update the state. We do this because React detects state changes more reliably when a new object reference is provided, which triggers a re-render and updates the UI with the new name. However, directly modifying tempData.name also modifies the original state object, so this approach works but is not considered a best practice in React.

Output:

Name: Utsav

City: Muzaffarpur

Country: India

This component stores a user's name and address in state using `useState`. When the user types in the input field, `handleName` updates the name property and calls `setData` with a copied object, causing React to re-render and display the updated name. However, it directly modifies the existing state object before copying it, which is not the recommended React pattern.

Now, suppose we want to change the nested part of the object, then? See below example:

App.jsx:

```
src > App.jsx > default
1  import React, { useState } from 'react'
2  function App() {
3    const [data, setData] = useState({
4      name: 'Aditya',
5      address: {
6        city: 'Muzaffarpur',
7        country: 'India'
8      }
9    })
10   const handleCity=(city)=>{
11     data.address.city = city;
12     setData({...data, address:{...data.address,city}})
13   }
14   return (
15     <div>
16       <input type='text' placeholder='Update city name'
17         onChange={(event)=>handleCity(event.target.value)}/>
18       <h2>Name: {data.name}</h2>
19       <h2>City: {data.address.city}</h2>
20       <h2>Country: {data.address.country}</h2>
21     </div>
22   )
23 }
24 export default App
```

This component stores a user's name and address in state. When the user types a new city name in the input field, the `handleCity` function updates the city property inside the nested address object. The `setData({...data, address: {...data.address, city}})` statement creates new copies of both the data object and the nested address object while updating the city value. This is done so React receives a new state object and re-renders the component to display the updated city. However, the line `data.address.city = city` directly mutates the state and should be avoided; the spread operator update alone is sufficient.

Output:

Name: Aditya

City: Patna

Country: India

Updating Arrays in State:

How to approach this?

In React, arrays in state should be treated as immutable, meaning you should create a new array instead of modifying the existing one. When updating an array, React needs a new array reference to detect the change and re-render the component.

First understand, displaying the array in ReactJS using State.

App.jsx:

```
src > App.jsx > ...
1  import React, { useState } from 'react'
2
3  function App() {
4    const [data, setData] = useState([
5      'adi', 'uts', 'pihu'
6    ])
7    return (
8      <div>
9        {
10          data.map((item, index)=>(
11            <h3 key={index}>{item}</h3>
12          ))
13        }
14      </div>
15    )
16  }
17  export default App
```

key is not mandatory for the code to run, but React will show a warning if it's missing in a list.

It helps React identify and efficiently update, add, or remove list items during re-rendering.

Output:

adi

uts

pihu

This component uses the useState hook to store an array of names ('adi', 'uts', and 'pihu') in the data state variable. Inside the JSX, the map() method is used to loop through each item in the array and render it as an <h3> element. The key={index} prop gives each rendered element a unique identifier, helping React efficiently update the UI when the list changes. As a result, all the names in the array are displayed on the screen as separate headings.

Now, I want that when I type a new name in the input field, the last name of the array shall get updated.

App.jsx:

```
src > App.jsx > App
1  import React, { useState } from 'react'
2
3  function App() {
4    const [data, setData] = useState([
5      'adi', 'uts', 'pihu'
6    ])
7    const handleUser=(name)=>{
8      data[data.length-1] = name;
9      setData([...data])
10   }
11   return (
12     <div>
13       <input type='text' placeholder='Enter last user name'
14         onChange={(e)=>handleUser(e.target.value)}/>
15       {
16         data.map((item, index)=>(
17           <h3 key={index}>{item}</h3>
18         ))
19       }
20     </div>
21   )
22 }
23 export default App
```

Output:

adi

uts

nihu

This component stores an array of names in state and displays them using `map()`. When the user types in the input field, the `handleUser` function updates the last element of the array with the entered value. After modifying the array, `setData([...data])` creates a new array copy and updates the state, causing React to re-render and display the updated last user's name.

useActionState Hook:

useActionState is a React Hook (introduced with modern React and commonly used with React Server Actions frameworks like Next.js) that helps manage the state resulting from a form action.

Syntax:

```
const [state, formAction, isPending] = useActionState(  
  action,  
  initialState  
);
```

When should you use it?

1. Form submissions
2. Server actions
3. Validation Results

useActionState vs useState:

Feature	useState	useActionState
General UI state	✓	✗
Form submissions	⚠ Manual handling	✓
Previous state passed automatically	✗	✓
Pending state	✗	✓
Works naturally with forms	✗	✓
Server Actions integration	✗	✓

Let's first create a basic form template:

App.jsx:

```
src > App.jsx > App  
1  import React from 'react'  
2  function App() {  
3    return (  
4      <div>  
5        <form action="">  
6          <input type='text' placeholder='Enter name' name='name' />  
7          <br/><br/>  
8          <input type='password' placeholder='Enter password' name='password' />  
9          <br/><br/>  
10         <button>Submit</button>  
11       </form>  
12     </div>  
13   )  
14 }  
15 export default App
```

Output:

Now, introducing useActionState, to take the input, and show it on console.

App.jsx:

```
src > App.jsx > App
1 import React, { useActionState } from 'react'
2 function App() {
3
4   const handleSubmit = (previousData,formData) => {
5     let name = formData.get('name');
6     let password = formData.get('password');
7     console.log("handleSubmit called", name, password);
8   }
9
10  const [data,action,pending] = useActionState(handleSubmit, undefined)
11
12  return (
13    <div>
14      <form action={action}>
15        <input type='text' placeholder='Enter name' name='name' />
16        <br/><br/>
17        <input type='password' placeholder='Enter password' name='password' />
18        <br/><br/>
19        <button>Submit</button>
20      </form>
21    </div>
22  )
23 }
24 export default App
```

useActionState registers handleSubmit as the form action, and when <form action={action}> is submitted, React automatically calls handleSubmit with the form data.

handleSubmit is the action function that runs when the form is submitted. It receives the form data, extracts the name and password, and logs them.

useActionState(handleSubmit, undefined) connects handleSubmit to React's action system and returns data (state), action (submit handler for the form), and pending (loading status).

This React component uses useActionState to handle form submissions. When the user submits the form, the action function triggers handleSubmit, which receives the form data, extracts the name and password values, and logs them to the console. useActionState also provides data (state) and pending (loading status), though they aren't used here.

Output:

Elements Console Sources Network Performance Memory Application Security Lighthouse

test

[debug-metadata] enter inject script test

[YD-CS-SUBTITLE] sidebar.content 已挂载 {"href":"http://localhost:5173/","readyState":"complete"}

handleSubmit called Aditya Password

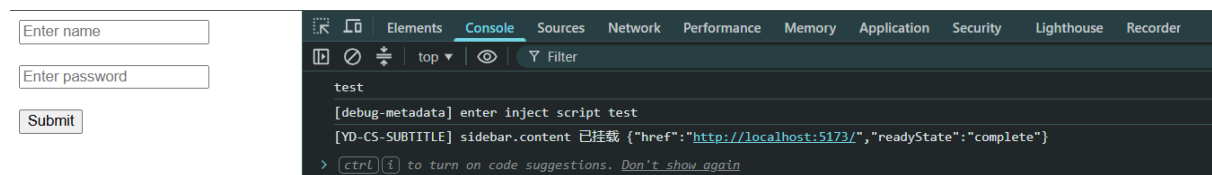
Now, suppose in the same form, we want that there shall be a waiting period before submitting the form once the user clicked the “submit” button.

App.jsx:

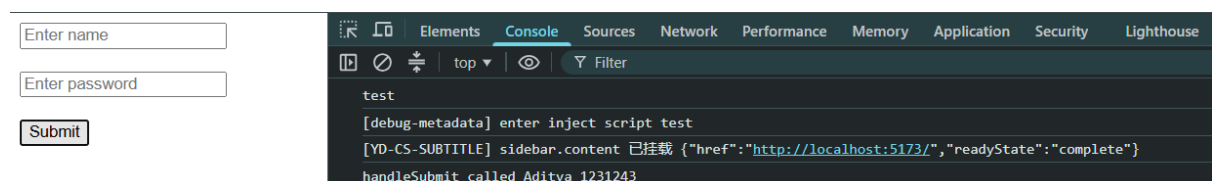
```
src > App.jsx > App
1  import React, { useActionState } from 'react'
2  function App() {
3
4      const handleSubmit = async(previousData,formData) => {
5          let name = formData.get('name');
6          let password = formData.get('password');
7          await new Promise(res=>setTimeout(res,2000));
8          console.log("handleSubmit called", name, password);
9      }
10
11     const [data,action,pending] = useActionState(handleSubmit, undefined)
12
13     return (
14         <div>
15             <form action={action}>
16                 <input type='text' placeholder='Enter name' name='name' />
17                 <br/><br/>
18                 <input type='password' placeholder='Enter password' name='password' />
19                 <br/><br/>
20                 <button>Submit</button>
21             </form>
22         </div>
23     )
24 }
25 export default App
```

Just added the promise there.

Output: when loads.



Output: when submitted we can see we have to wait for 2 seconds, before it get reflected in the console.

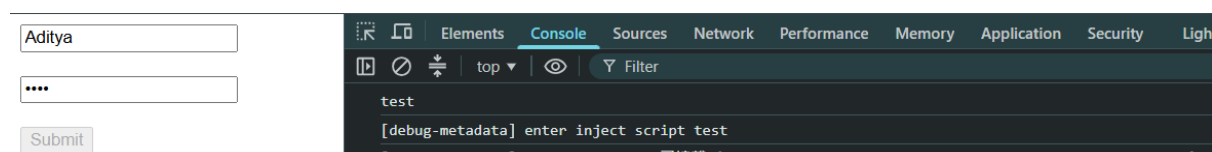


Now, suppose we want that when the user clicks the submit button, the submit button shall be disabled till the from is not submitted (here for 2 seconds in above case).

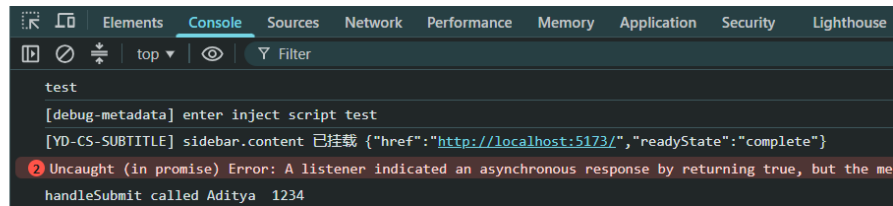
App.jsx: rest code be same as above for App.jsx.

```
19     <br/><br/>
20     <button disabled={pending}>Submit</button>
21 </form>
```

Output: the moment we submitted the button got disabled.



Output: the moment the form got submitted and shown on the console, it get enabled as well.



useld Hook:

useld is a React Hook that generates a unique and stable ID for a component. It is mainly used for accessibility by connecting form elements like labels and inputs.

Basic Syntax:

```
const id = useld();
```

Why do we need useld?

Consider a form with multiple inputs:

```
<label htmlFor="name">Name</label>  
<input id="name" />
```

This works fine for one input. But if the same component is rendered multiple times:

```
<MyInput />  
<MyInput />
```

Both inputs would have the same ID (name), which is invalid HTML. useld solves this by generating unique IDs automatically.

Advantages:

- Stable across re-renders.
- Works correctly with Server-Side Rendering (SSR).
- Guaranteed unique within the React tree.

Also,

One of the biggest reasons React introduced useld is to support Server-Side Rendering.

Without useld: `const id = Math.random();`

Server: 0.123

Client: 0.456

React detects different HTML and may show hydration warnings.

With useld: `const id = useld();`

React generates matching IDs on both server and client

Example: a basic example showing that the ids generated are unique.

App.jsx:

```
src > App.jsx > App
1  import React, { useId } from 'react'
2
3  function App() {
4    const name = useId();
5    const terms = useId();
6    const condition = useId();
7    return (
8      <div>
9        <h1>{name}</h1>
10       <h1>{terms}</h1>
11       <h1>{condition}</h1>
12     </div>
13   )
14 }
15
16 export default App
```

Output:

__r_0__

__r_1__

__r_2__